MEMORIA TÉCNICA — DISNEY UNIVERSE

Catálogo y trivial interactivo de personajes Disney

Proyecto grupal del bootcamp Full Stack — Grupo 1

Ainara · Andrés Felipe · Carmen · Jon Mikel

Tecnologías: HTML5, CSS3, JavaScript (módulos ES), JSON y Disney API

1. Descripción del proyecto

Disney Universe es una aplicación web multipágina orientada a la exploración interactiva del universo de personajes Disney. Permite consultar un catálogo de personajes, filtrar por diferentes categorías (nombre, películas y videojuegos), ordenar alfabéticamente en ambos sentidos, gestionar una lista personalizada de favoritos persistente en el navegador y poner a prueba los conocimientos mediante un juego de trivial con sistema de puntuación.

1.1 Objetivos de aprendizaje

- Construir una aplicación web multipágina empleando HTML5, CSS3 y JavaScript vanilla (ES Modules), sin frameworks de interfaz.
- Estructurar la lógica del dominio en clases JavaScript y módulos con responsabilidades diferenciadas.
- Separar la obtención de datos, la lógica de negocio y la presentación visual (core, ui, utils).
- Manejar peticiones asíncronas a APIs REST mediante fetch e integrar paginación.
- Conservar preferencias del usuario entre sesiones mediante localStorage.
- Colaborar con Git, gestionar ramas y resolver conflictos de integración.

1.2 Alcance

Dentro del alcance	Fuera del alcance
Consulta dinámica del catálogo mediante paginación.	Autenticación o registro real con contraseña (se simula).
Juego de trivial: identificar personajes por imagen.	Ranking persistente de jugadores y puntuaciones.
Filtros por nombre, películas y videojuegos; ordenación A-Z / Z-A.	Ficha ampliada individual de personajes.
Guardado y consulta de favoritos con localStorage.	Favoritos asociados a cuenta de usuario.

2. Equipo y reparto de tareas (Grupo 1)

Aunque el desarrollo se caracterizó por una estrecha colaboración entre integrantes, las responsabilidades principales quedaron distribuidas de la siguiente manera:

Miembro	Responsabilidades principales
Ainara	Codiseño del CSS, renderizado colaborativo de tarjetas y desarrollo del módulo de filtros, ordenación y favoritos.
Andrés Felipe	Codiseño y maquetación interactiva de tarjetas de personajes (cards) y colaboración en su renderizado.
Carmen	Integración de la API Disney, paginación, persistencia con localStorage, arquitectura de clases en core y pantalla de favoritos.
Jon Mikel	Diseño y programación de la lógica del trivial: preguntas, validación de respuestas y puntuación final.

3. Requisitos de la aplicación

3.1 Requisitos funcionales

ID	Requisito	Ubicación orientativa
RF-01	Consultar personajes recuperados desde la API pública.	core/ · utils/
RF-02	Navegar por el catálogo mediante paginación.	src/html/catalog.html
RF-03	Filtrar por nombre, películas o videojuegos.	src/html/catalog.html
RF-04	Ordenar personajes A-Z / Z-A.	core/ · ui/
RF-05	Marcar o desmarcar favoritos con persistencia local.	core/favorites.js · localStorage.js
RF-06	Consultar una vista de favoritos.	src/html/favorites.html
RF-07	Jugar una partida de trivial de 5 preguntas con respuesta múltiple.	src/html/game.html · core/game.js
RF-08	Mostrar la puntuación al terminar.	src/html/gameResult.html · core/gameResult.js

3.2 Requisitos no funcionales

- RNF-01. JavaScript modular sin frameworks: uso de módulos ES (import/export).
- RNF-02. Separación de responsabilidades: carpetas core/, ui/, utils/, api/, html/ y css/.
- RNF-03. CSS organizado por vistas: index, catálogo y juego.
- RNF-04. Persistencia de favoritos mediante localStorage.

4. Estructura y arquitectura del proyecto

```
disney-universe/
├── index.html
├── README.md
└── src/
  ├── api/apiKeys.js
  ├── core/
  │  ├── Catalog.js
  │  ├── Character.js
  │  ├── cards.js
  │  ├── favorites.js
  │  ├── game.js
  │  ├── gameResult.js
  │  ├── home.js
  │  └── localStorage.js
  ├── css/ (catalog.css, game.css, index.css)
  ├── data/
  ├── html/ (catalog.html, favorites.html, game.html, gameResult.html)
  ├── img/ · media/
  ├── ui/ (redender.js — verificar nombre real)
  └── utils/ (api.js, errors.js, functions.js)
```

4.1 Modelo de clases y programación orientada a objetos

La aplicación organiza parte de sus datos y de la lógica del dominio mediante clases JavaScript. Según la estructura documentada, Character.js representa los personajes y Catalog.js se encarga de gestionar el catálogo. La separación permite diferenciar los datos de cada personaje de las operaciones realizadas sobre el conjunto.

Módulo / clase	Responsabilidad documentada	Pendiente de completar con código real
Character.js	Modelo de los datos de un personaje, a partir de la respuesta de la API.	Atributos, constructor y métodos concretos.
Catalog.js	Gestión del catálogo y de los personajes disponibles.	Métodos de carga, filtros y ordenación; relaciones con Character.
game.js y gameResult.js	Lógica de preguntas, validación y resultados del trivial.	Funciones públicas, estados y cálculo exacto de puntos.

4.2 Flujo de datos

La información de los personajes procede de la API pública **Disney API.** La aplicación es frontend y no dispone de backend ni de base de datos relacional propia. Usa módulos JavaScript para las interacciones y localStorage para conservar los favoritos entre sesiones.

5. Diseño de interfaz y experiencia de usuario (UI/UX)

La propuesta visual busca transmitir el carácter alegre, lúdico y reconocible del universo Disney. Se apuesta por una presentación divertida y dinámica, con protagonismo de las imágenes de los personajes y una separación clara entre explorar el catálogo, consultar favoritos y jugar al trivial.

La interfaz se organiza mediante tarjetas de personajes: este formato favorece el reconocimiento visual y facilita la exploración de grandes colecciones. Los controles de búsqueda, filtros y ordenación permiten acotar el contenido sin obligar al usuario a revisar todo el catálogo.

En el trivial, el uso de imágenes y respuestas de opción múltiple prioriza una interacción directa y fácil de entender. La pantalla de resultado ofrece cierre a cada partida mediante la puntuación obtenida. El tono visual busca ser accesible y entretenido, sin recargar la consulta de información.





6. Gestión del trabajo y seguimiento

6.1 Organización colaborativa

La división por funcionalidades permitió avanzar en paralelo en integración de datos, visualización, filtros, favoritos y trivial. La integración se coordinó mediante control de versiones Git. Las responsabilidades principales se recogen en la siguiente tabla

Bloque de trabajo	Responsable(s)	Evidencia pendiente
Consumo de API, paginación y modelos	Carmen	Fechas, tarjetas o commits
Tarjetas y maquetación de catálogo	Ainara y Andrés Felipe	Fechas, tarjetas o commits
Filtros, ordenación y favoritos	Ainara y Carmen	Fechas, tarjetas o commits
Preguntas y puntuación del trivial	Jon Mikel	Fechas, tarjetas o commits
Integración de ramas y resolución de conflictos	Equipo	Pull requests o incidencias relevantes

7. Decisiones técnicas y desafíos

7.1 Cambio de la idea inicial

En un primer momento, el grupo valoró desarrollar una plataforma dedicada al reino animal y la fauna. Tras explorar varias APIs públicas, se decidió que las fuentes examinadas no proporcionaban el volumen y la estructura de datos necesarios para el enfoque visual e interactivo previsto. Por ello, se optó por Disney API, que ofrece personajes, imágenes e información de películas y videojuegos adecuada para el catálogo y el trivial.

7.2 Control de versiones y dificultades en Git

El principal reto de colaboración fue la gestión de Git. El trabajo simultáneo sobre archivos de lógica e interfaz produjo conflictos de fusión al integrar ramas con funcionalidades paralelas. La respuesta del equipo consistió en revisar conjuntamente el código, coordinar pull requests y delimitar mejor la responsabilidad de cada módulo antes de integrar cambios.

8. Ampliaciones futuras y conclusiones

8.1 Posibles mejoras

- Pruebas unitarias para el trivial y para la transformación de datos procedentes de la API.
- Nuevas mecánicas del trivial: temporizador y niveles de dificultad.
- Mejoras UX: transiciones y estados de carga más visibles durante las consultas a la API.

8.2 Conclusiones

Disney Universe reúne en una misma aplicación los conocimientos adquiridos sobre JavaScript modular, programación orientada a objetos, consumo asíncrono de APIs, persistencia local y construcción de interfaces dinámicas. El cambio de temática y los conflictos de integración mostraron la importancia de evaluar la viabilidad de los datos y coordinar el desarrollo en equipo. Como continuidad, el proyecto podría reforzar las pruebas automatizadas y pulir la experiencia del juego y del catálogo.